Task Profiling and Draft Model Selection for Accelerating Distributed Speculative Decoding

Jialin Dong, Yaodan Xu, Tan Chen, Sheng Zhou, Zhisheng Niu
Department of Electronic Engineering, Tsinghua University, Beijing, China
djl23@mails.tsinghua.edu.cn, xyd21@mails.tsinghua.edu.cn, chent21@mails.tsinghua.edu.cn,
sheng.zhou@tsinghua.edu.cn, niuzhs@mail.tsinghua.edu.cn

***Abstract*—Speculative Decoding (SD) has emerged as an effective strategy to accelerate Large Language Model (LLM) inference by leveraging lightweight draft models to speculate candidate tokens. The heterogeneity of devices in edge-cloud systems provides a unique opportunity to deploy task-specialized draft models, leveraging each device's distinctive computational profile to maximize speculative efficiency. However, existing draft model selection strategies often neglect the impact of task profiles. To address this gap, we propose D-PROMISE (Distributed task PROfiling based draft Model Selection Engine), a framework that minimizes end-to-end inference latency through request-level optimization. Specifically, D-PROMISE categorizes the task into four types based on its task entropy and reasoning intensity. The engine maintains statistical records of draft model proficiencies across these categories to predict task-specific acceptance rates. Simultaneously, D-PROMISE monitors real-time edge device dynamics, including computational loads and wireless channel conditions. Taking task profiling and dynamic device states into consideration, the scheduler identifies the optimal device and speculative length for each request. Trace-driven simulations, parameterized with empirical hardware and network profiles, show that D-PROMISE achieves up to a 45% increase in inference speed compared to task-agnostic speculative decoding, demonstrating the necessity of integrating task profiling and device dynamics for speculative inference.**

I. Introduction

The rapid evolution of LLMs has unlocked immense potential across diverse domains [1]. However, their deployment in interactive applications is severely constrained by the high end-to-end latency of auto-regressive decoding. Speculative Decoding (SD) has emerged as a transformative acceleration paradigm. Unlike traditional decoding, SD decomposes the inference process into a propose-then-verify pipeline: it leverages a lightweight *draft model* to rapidly generate a sequence of K candidate tokens, which are then verified and corrected in parallel by the *target model* in a single forward pass. This mechanism mathematically guarantees that generated outputs are identical to those of the target model through provably equivalent sampling algorithms [2], [3]. The efficacy of this paradigm heavily relies on the *acceptance rate* (α)—the probability that speculated tokens accepted by the target model. Crucially, the theoretical speedup is a function of both α and the relative computational overhead between the two models; a higher α increases the expected number of valid tokens confirmed in a single pass, thereby significantly accelerating the inference process.

This work is supported in part by the Open Fund of State Key Laboratory of Intelligent Green Vehicle and Mobility, Tsinghua University, and the project of Tsinghua University-Toyota Joint Research Center for AI Technology of Automated Vehicle.

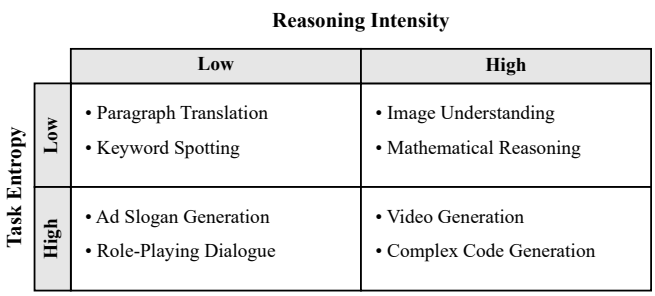


Fig. 1: Task profiling based on task entropy and reasoning intensity.

Previous SD studies primarily focus on centralized deployments, co-locating a static, pre-trained draft model with the target model in the cloud. However, this paradigm is inherently limited in scalability and adaptability within the increasingly prevalent edge-cloud landscape. In such environments, a massive pool of heterogeneous edge devices, ranging from smartphones to PCs, often host locally fine-tuned draft models tailored to specific user-end tasks. Storing all these models on a single cloud server is impractical due to the massive memory requirements and the potential privacy risks associated with uploading local model weights. To address these challenges, we leverage a distributed edge-cloud architecture where the resource-intensive target model remains on the cloud server, while the pre-existing, domain-specific draft models are maintained at the edge. This collaborative strategy not only preserves data privacy but also exploits the specialized proficiencies and idle computational resources of edge nodes. Nevertheless, effectively utilizing these distributed resources is non-trivial: edge devices exhibit significant variance in hardware capabilities, and their performance is also subject to stochastic local workloads and fluctuating wireless channel conditions. Consequently, the problem lies in coordinating task profiles with these dynamic system states to optimize draft model selection for inference acceleration.

Recent studies highlight that task characteristics significantly impact speculative decoding efficacy. Specifically, draft models exhibit distinct domain preferences [4], while acceptance rates correlate strongly with drafted token entropy [5]

and reasoning intensity [6]. Synthesizing these insights, we categorize requests based on two dimensions: task entropy and reasoning intensity, which are further formulated in Section II-B. Task entropy is quantified by the average information entropy of the target model's output token distribution, while reasoning intensity is characterized via logical depth, the number of sequential reasoning steps required to reach the final answer, as illustrated in Fig. 1. D-PROMISE employs a lightweight classifier to map incoming tasks to these profiles, leveraging historical proficiency logs—maintained via multi-armed bandit learning—as task-specific priors for optimization.

Beyond task semantics, edge-cloud SD is constrained by device dynamics. Ref. [7] identifies a critical memory saturation threshold, beyond which resource contention triggers nonlinear latency degradation. Additionally, uplink transmission of speculated tokens incurs communication overhead modulated by time-varying wireless channel quality. To address these, we propose D-PROMISE, a framework for request-level optimization in edge-cloud environments.

Our contributions are summarized as follows:

- **Multi-Dimensional Task Profiling:** We propose a taxonomy categorizing incoming prompts by task entropy and reasoning intensity. By mapping tasks to four types, we enable granular, context-aware prediction of acceptance rates across heterogeneous models.
- **Task-Adaptive Draft Model Selection Framework:** We develop D-PROMISE to perform request-level optimization. By aligning task characteristics with real-time computational loads and channel conditions, the engine jointly optimizes the draft device and speculative length. Trace-driven simulations show up to **45%** speedup over task-agnostic SD baselines.

II. System Architecture and Problem Formulation

In this section, we present the architecture of D-PROMISE, a collaborative inference framework designed for distributed environments characterized by heterogeneous edge devices and a centralized high-performance server. While D-PROMISE is applicable to various scenarios (e.g., smart factories or interactive robotics), we exemplify our framework using a smart office scenario. We first detail the physical entities and the logical workflow of the task profiling and draft model selection mechanism. Subsequently, we mathematically model the latency dynamics under fluctuating device conditions and formulate the optimization problem for latency minimization.

A. System Overview

As illustrated in Fig. 2, the system consists of two primary physical entities: the target server and a pool of heterogeneous devices. Target server is a centralized high-performance computing node (e.g., an on-premise workstation) that acts as the system's core. It logically contains two functional modules: 1) GPU node: the compute engine loading the large-scale target model. It is responsible for parallel token verification and fallback generation to guarantee lossless output. 2) D-PROMISE: the control engine deployed on the server. It integrates the task analyzer for task profiling and the adaptive scheduling core including the resource monitor and latency-predictive scheduler. Heterogeneous device pool is a set of N worker devices, denoted as $\mathcal{D} = \{d_1, d_2, \ldots, d_N\}$. These devices (e.g., laptops, tablets, or smartphones) function as potential draft models. Each device d_i loads a lightweight draft model $\mathcal{M}_{draft}^{(i)}$.

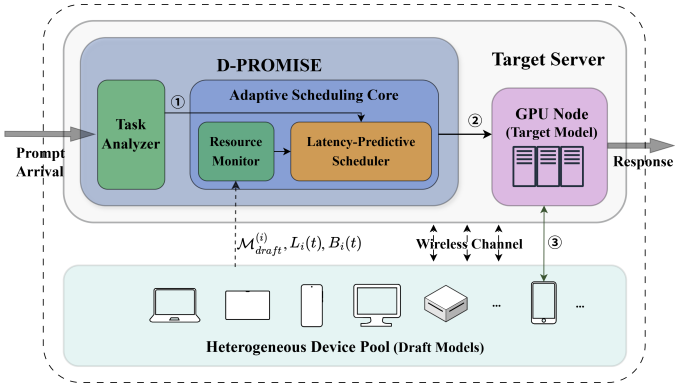


Fig. 2: Architecture of the D-PROMISE framework.

The inference process proceeds in three phases (Fig. 2): First, the task analyzer profiles the prompt while the resource monitor aggregates the model profile $\mathcal{M}_{draft}^{(i)}$ and dynamic real-time conditions, specifically the computational load $L_i(t)$ and channel bandwidth $B_i(t)$. Second, the scheduler executes D-PROMISE to select the optimal device d^* and length γ^*. Finally, the server and d^* execute speculative decoding iteratively.

B. Task Profiling

As shown in Fig. 2, the task analyzer outputs a task-type label τ by classifying each request along two attributes: *task entropy* (the inherent uncertainty of the next-token distribution) and *reasoning intensity* (the depth of multi-step reasoning required).

Task entropy is quantified by the average entropy of the target model's output distribution. For a sequence of length T, the score is $\mathcal{H}(x) \triangleq \frac{1}{T}\sum_{t=1}^{T} H(p_t(\cdot))$, where $H(p_t)$ is the Shannon entropy at step t. A request is High-Entropy (HE) if $\mathcal{H}(x) \geq \theta_{\mathrm{H}}$; otherwise it is categorized as Low-Entropy (LE). The threshold θ_{H} is selected offline on a calibration set to ensure stable regime boundaries.

Reasoning intensity is characterized via logical depth, i.e., the number of sequential reasoning steps required to reach the final answer. Operationally, a lightweight depth estimator is invoked to produce a stepwise plan consisting of $K(x)$ atomic reasoning steps for input x.[1] The reasoning-intensity score is defined as

$$\mathcal{R}(x) \triangleq K(x),$$

[1]In practice, $K(x)$ can be extracted by enforcing a numbered step format and counting the steps, without requiring the full final answer content.

and a request is classified as High-Intensity (HI) if $\mathcal{R}(x) \geq \theta_{\mathrm{R}}$; otherwise it is classified as Low-Intensity (LI). Similar to θ_{H}, θ_{R} is chosen offline to provide a robust separation between shallow and deep-reasoning workloads.

In practice, the task analyzer can be instantiated as a lightweight deep neural classifier that directly predicts the task-type label τ (or the two binary attributes) from the prompt text. Its inference time is typically at the millisecond scale, which is negligible compared to the cumulative seconds-level latency of end-to-end LLM decoding; accordingly, the task analyzer's inference latency is ignored throughout this paper. Moreover, since downstream acceptance-rate statistics are computed conditioned on the task analyzer's predicted labels, any misclassification is implicitly reflected in the empirically estimated acceptance matrix $\boldsymbol{\alpha}$; that is, the impact of classifier errors is absorbed into the acceptance-rate measurements used by the scheduler.

C. Latency Analysis and Problem Formulation

The primary objective of D-PROMISE is to minimize end-to-end inference latency through task profiling and draft model selection. In this formulation, the initial prompt upload and final response transmission times are omitted, as they are negligible compared to the generation latency. To capture device heterogeneity, real-time conditions at time slot t are represented by an aggregate state variable $\boldsymbol{S}(t) \triangleq \{\boldsymbol{B}(t), \boldsymbol{L}(t), \boldsymbol{\alpha}\}$. Here, $\boldsymbol{B}(t)$ denotes the vector of device-specific real-time bandwidths, while $\boldsymbol{L}(t)$ represents the fluctuating device-specific computational load. Furthermore, $\boldsymbol{\alpha}$ constitutes the task-dependent acceptance-rate matrix. This alignment information is assumed to be encapsulated within the draft model profile $\mathcal{M}_{draft}^{(i)}$ and made available as prior knowledge derived from historical statistics.

The physical wall-clock time required for a single speculative iteration is denoted as T_{iter}. When device d_i is selected with a draft length γ, the latency is decomposed into three components: draft-model inference latency, communication latency, and target-model verification latency. The drafting latency is modeled as $\gamma \cdot \Phi_i(L_i(t))$, where $\Phi_i(\cdot)$ is a device-specific profiling function that maps the current load $L_i(t)$ to the per-token generation time. The communication latency accounts for transmitting the payload of γ token distribution and IDs (each of size S_{token}) and protocol headers (S_{header}) over the time-varying channel $B_i(t)$, together with other delays captured by δ_i. Combining these with the constant verification time T_{verify} on the target server, the total iteration latency is formulated as:

$$T_{iter}(d_i, \gamma \mid \boldsymbol{S}(t)) = \gamma \cdot \Phi_i(L_i(t)) + T_{verify} + \left(\frac{\gamma S_{token} + S_{header}}{B_i(t)} + \delta_i \right). \quad (1)$$

While T_{iter} captures the physical cost, the acceleration gain depends on the quantity of valid tokens generated. Let $\alpha_{i,\tau}$ denote the acceptance probability when device d_i serves a request of task type τ. Assuming token verification outcomes are i.i.d., the number of accepted tokens follows a truncated geometric distribution. Accordingly, the expected number of valid tokens per iteration $\mathbb{E}[N_{acc}]$ is derived as:

$$\mathbb{E}[N_{acc} \mid d_i, \tau, \gamma] = \sum_{k=0}^{\gamma} \alpha_{i,\tau}^k = \frac{1 - \alpha_{i,\tau}^{\gamma+1}}{1 - \alpha_{i,\tau}}. \quad (2)$$

To evaluate inference efficiency, the expected latency per valid token $\bar{T}_{valid}$ is defined as the ratio of the iteration cost to the expected valid tokens:

$$\bar{T}_{valid}(d_i, \gamma \mid \tau, \boldsymbol{S}(t)) = \frac{T_{iter}(d_i, \gamma \mid \boldsymbol{S}(t))}{\mathbb{E}[N_{acc} \mid d_i, \tau, \gamma]}. \quad (3)$$

Consequently, the orchestration problem is formulated as finding the optimal device-configuration pair $\{d^*, \gamma^*\}$ that minimizes this metric under the instantaneous system state:

$$\begin{aligned} \min_{d_i \in \mathcal{D}, \gamma \in \mathbb{Z}^+} \quad & \bar{T}_{valid}(d_i, \gamma \mid \tau, \boldsymbol{S}(t)) \\ \text{s.t.} \quad & 1 \leq \gamma \leq \gamma_{max}, \quad d_i \in \mathcal{D}_{active}(t). \end{aligned} \quad (4)$$

Since $\bar{T}_{valid}$ fluctuates dynamically with both the task type τ and the system state $\boldsymbol{S}(t)$, this optimization requires a lightweight per-request solving strategy, which is detailed in Section III.

III. Task-Aware Draft Model Selection Algorithm

The optimization problem in Eq. (4) requires identification of the optimal device-configuration pair $\{d^*, \gamma^*\}$ that minimizes the expected latency per valid token. A key challenge is that the optimal solution is non-stationary, shifting with the task category of the request (denoted as τ) and the instantaneous environmental state $\boldsymbol{S}(t)$. To address this, D-PROMISE is introduced as a request-level scheduling policy operating over a finite, pre-defined set of four task categories $\mathcal{T}$. Upon arrival, each prompt is mapped to a specific type $\tau \in \mathcal{T}$ by the lightweight classifier. The resulting label τ is then used to index the task-dependent acceptance matrix $\boldsymbol{\alpha}$ and guide the joint selection of the drafting device and draft length $\{d^*, \gamma^*\}$ for each request.

A. Device Profiling and Initialization

To minimize online scheduling overhead, profiling is decoupled from real-time decision-making in D-PROMISE. Two distinct knowledge bases are maintained to support the rapid evaluation of the objective metric $\bar{T}_{valid}$. First, the task alignment is handled via the acceptance rate matrix $\boldsymbol{\alpha}$. As defined in Section II-C, this static prior maps the identified task category τ to the specific acceptance probability of each device, serving as the basis for effective latency estimation. Second, the physical drafting capability is characterized by device-specific profiling functions $\Phi_i(\cdot)$. Obtained via offline regression, these functions allow the scheduler to instantly estimate the per-token drafting latency $t_{draft} = \Phi_i(L_i(t))$ given the current load. This pre-computation eliminates the need for intrusive real-time benchmarking, ensuring that draft model selection latency remains negligible.

Algorithm 1 D-PROMISE: Distributed task PROfiling based draft Model Selection Engine

Require: task category $\tau \in \mathcal{T}$, system state $\boldsymbol{S}(t) = \{\boldsymbol{B}(t), \boldsymbol{L}(t), \boldsymbol{\alpha}\}$, constants $S_{token}, S_{header}, \gamma_{max}, \delta_i$, edge auto-regressive inference latency T_{auto}
Ensure: optimal decision $\{d^*, \gamma^*\}$

Initialize: $\mathcal{L}_{min} \leftarrow T_{auto},\ d^* \leftarrow$ Edge, $\gamma^* \leftarrow 0$
for each device $d_i \in \mathcal{D}_{active}$ **do**
  Retrieve prior $\alpha_{i,\tau}$ from $\boldsymbol{\alpha}$, B_i and L_i from $\boldsymbol{S}(t)$;
  for $\gamma = 1$ to γ_{max} **do**
    $t_{comm} \leftarrow (\gamma \cdot S_{token} + S_{header})/B_i + \delta_i$;
    $T_{iter} \leftarrow \gamma \cdot \Phi_i(L_i) + t_{comm} + T_{verify}$;
    $\mathbb{E}[N_{acc}] \leftarrow (1 - \alpha_{i,\tau}^{\gamma+1})/(1 - \alpha_{i,\tau})$;
    $\mathcal{L}_{curr} \leftarrow T_{iter}/\mathbb{E}[N_{acc}]$;
    if $\mathcal{L}_{curr} < \mathcal{L}_{min}$ **then**
      $\mathcal{L}_{min} \leftarrow \mathcal{L}_{curr},\ d^* \leftarrow d_i,\ \gamma^* \leftarrow \gamma$;
    else if $\mathcal{L}_{curr} == \mathcal{L}_{min}$ **and** $L_i < L_{d^*}$ **then**
      $d^* \leftarrow d_i,\ \gamma^* \leftarrow \gamma$;
    end if
  end for
end for
return $\{d^*, \gamma^*\}$;

B. Online Scheduling Strategy

The execution logic of D-PROMISE is formalized in Algorithm 1. Upon receiving a request, the task analyzer identifies the task category τ and captures the instantaneous system state $\boldsymbol{S}(t)$. To guarantee system robustness, the latency baseline $\mathcal{L}_{min}$ is initialized to T_{auto}, representing the standard autoregressive inference time on the edge server. This establishes an implicit fallback mechanism: if network degradation or high device contention renders distributed speculation inefficient (i.e., the computed $\bar{T}_{valid}$ for all candidates exceeds T_{auto}), the system automatically defaults to local edge-only inference.

The core scheduling loop iterates through all active devices in $\mathcal{D}_{active}$. For each candidate, D-PROMISE evaluates the expected latency per valid token $\bar{T}_{valid}$ across the feasible draft length space $\gamma \in [1, \gamma_{max}]$. This step quantifies the trade-off between amortizing the fixed communication overhead (favoring larger γ) and accumulating drafting latency. Crucially, by incorporating the task-specific acceptance rate $\alpha_{i,\tau}$, the metric penalizes devices that are semantically misaligned with the current task. This mechanism prevents the selection of devices that possess high raw computational throughput but lack the specific domain expertise required for efficient speculation. Finally, to resolve ties in occasional homogeneous environments, D-PROMISE prioritizes the device with the lower current load $L_i(t)$, thereby promoting load balancing before dispatching the optimal configuration $\{d^*, \gamma^*\}$.

The acceptance rate matrix $\boldsymbol{\alpha}$ serves as the critical alignment layer between diverse task profiles and the heterogeneous pool of draft models. To obtain and maintain these task-specific acceptance rates, a hybrid strategy involving offline initialization and online adaptation is considered. Initially, the matrix is populated using historical proficiency logs or pre-evaluated benchmarks that map specific device capabilities to the four task categories. To maintain the matrix's accuracy in dynamic environments, D-PROMISE can leverage the inherent verification step of speculative decoding, which naturally provides the ground truth acceptance count for every iteration without additional labeling overhead. By treating each inference request as a learning sample, the framework can iteratively refine its proficiency estimates using reinforcement learning concepts, such as Multi-Armed Bandit (MAB) updates or moving average statistics. This iterative feedback loop ensures that the matrix remains responsive to updates in local draft models, shifts in task distributions, or changes in edge device performance.

The computational complexity of the D-PROMISE algorithm is $O(N \cdot \gamma_{max})$, where N denotes the number of active candidates and γ_{max} represents the maximum draft length. In practical edge computing scenarios, the size N is naturally limited and γ_{max} is a bounded integer. Consequently, the execution time of the scheduling logic remains in the microsecond regime. Given that end-to-end inference typically spans the second-level timescale, this overhead is negligible, ensuring seamless real-time adaptation.

IV. Performance Evaluation

In this section, we evaluate the performance of D-PROMISE through trace-driven simulations. Our experiments aim to validate: (1) the efficacy of task profiling in exploiting the diversity of heterogeneous edge devices; (2) the significant inference latency speedup compared to heuristic baselines; and (3) the algorithmic robustness under volatile network conditions and computing load saturation.

A. Experimental Setup

A discrete-event simulator is developed to model the interaction between the target server and a pool of heterogeneous edge devices $\mathcal{D}$. The simulation incorporates realistic hardware profiles, stochastic channel models, and inference characteristics derived from empirical measurements. We assume the pool of edge devices $\mathcal{D}$ consists of three distinct devices: a smartphone, a tablet, and a laptop. The drafting capabilities are profiled based on representative open-source LLMs with parameter scales appropriate for each device's hardware constraints. As summarized in Table I, their drafting latencies (t_{draft}) and scale assumptions reflect empirical benchmarks [8].

The inference requests are divided into four types by task entropy and reasoning intensity where **LE/HE** denote low-/high-task-entropy inputs and **LI/HI** denote low-/high-reasoning-intensity requests. The core premise of our simulation is that the acceptance rate $\alpha_{i,\tau}$ is determined by the interaction between (i) device-constrained model capability and (ii) the intrinsic difficulty of tasks. Consistent with this premise, acceptance rates increase roughly with model strength. At the same time, HE tasks exhibit uniformly lower acceptance than LE tasks due to higher uncertainty and reduced target model agreement. HI tasks lead to lower acceptance rates

TABLE I: Simulation Parameters

Parameter	Value
Heterogeneous Device Profiling ($t^{(i)}_{draft}$)	
Smartphone (NPU, $\sim$ 500M)	4.0 ms/token
Tablet (Balanced, $\sim$ 3B)	10.0 ms/token
Laptop (High-Perf, $\sim$ 7B)	20.0 ms/token
Acceptance Rate Matrix $\boldsymbol{\alpha}$ (Phone, Tablet, Laptop)	
LE-LI	$[0.75,\ 0.78,\ 0.80]$
LE-HI	$[0.35,\ 0.50,\ 0.85]$
HE-LI	$[0.50,\ 0.55,\ 0.50]$
HE-HI	$[0.10,\ 0.30,\ 0.40]$
Target Server & Device Dynamics	
Target Model Latency (T_{AR})	100.0 ms/token
Verification Latency (T_{verify})	100.0 ms
Comm. Header (S_{header})	800 bits (100 Bytes)
Token Payload (S_{token})	8000 bits (1000 Bytes)
Downlink Bandwidth ($B(t)$)	$\mathcal{N}(\mu = 20, \sigma = 5)$ Mbps
Fixed latency (δ_i)	5.0 ms

than LI tasks due to insufficient thinking depth for lightweight models. This effect is most pronounced on the Smartphone, whose limited model capacity causes a sharp degradation under HE-HI, where both uncertainty and reasoning demands peak. In contrast, the Laptop's draft model is specialized for high reasoning intensity, which yields a noticeable advantage in LE-HI. The adopted numerical range of $\alpha \in [0.10, 0.85]$ is consistent with empirical acceptance probabilities reported in recent speculative decoding benchmarks across diverse datasets [9]–[11].

For the target server, we assume a 70B parameter model (e.g., Llama-3-70B scale) is loaded. Consistent with performance reports from production-grade inference engines (e.g., vLLM on NVIDIA A100/H100 clusters) [12], [13], we set the baseline autoregressive latency T_{AR} to 100 ms per token. We model the volatile wireless communication bandwidth $B(t)$ as a truncated Gaussian distribution ($\mu = 20, \sigma = 5$ Mbps) with a 800-bit protocol overhead and assume the fixed communication latency as 5ms. For computation load, we adopt the well-established nonlinear saturation model [14], where the latency penalty scales linearly in the under-utilized region ($L(t) < 0.6$) but exhibits exponential growth near saturation ($L(t) \to 1.0$) due to severe resource contention. Specifically, the latency penalty is then modeled as

$$\phi(L(t)) \triangleq \begin{cases} 1 + 0.5\,L(t), & L(t) < 0.6, \\ 1 + 0.5\,L(t) + \\ \exp(10(L(t) - 0.6)) - 1, & L(t) \geq 0.6, \end{cases}$$

B. Task-Aware Scheduling Speedup Analysis

To evaluate the effect of task profiling, we benchmark D-PROMISE against the standard non-speculative inference baseline and two representative SD strategies. The Edge-Only baseline executes the entire inference on the target server without speculative decoding. Laptop-Only represents a static compute-centric policy that delegates all drafting tasks to the device with the strongest on-device model. Task-Agnostic SD is an optimization-based baseline that selects the drafting

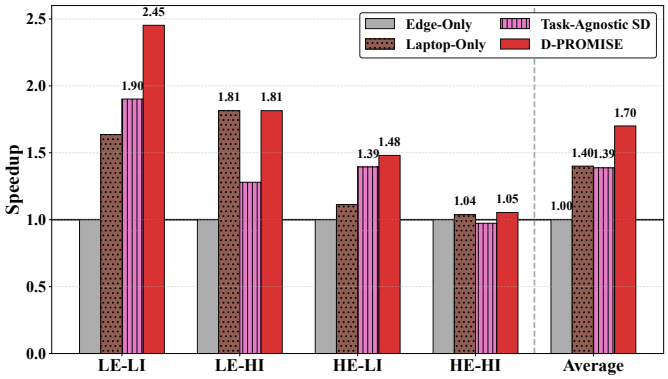


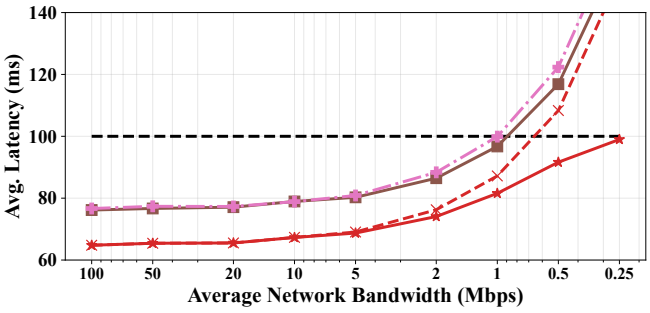
Fig. 3: Speedup comparison across different task types.

device using only the historical **average** acceptance rate $\bar{\alpha}_i$, ignoring the task-specific alignment of the current request τ.

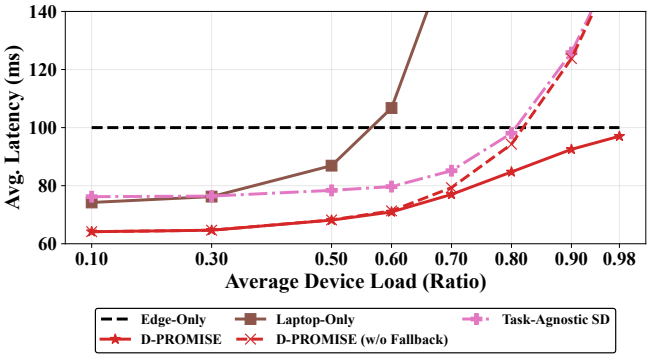
Based on the system configurations and device profiles established above, we conduct extensive trace-driven simulations to evaluate end-to-end latency under heterogeneous devices and workload regimes. Fig. 3 reports the inference speedup of each strategy relative to Edge-Only across four task types $\tau \in \{\text{LE-LI}, \text{LE-HI}, \text{HE-LI}, \text{HE-HI}\}$ and their average. Overall, D-PROMISE achieves the best aggregate performance, delivering an average speedup of 1.70, outperforming Laptop-Only and Task-Agnostic SD. This indicates that explicitly task profiling yields consistent end-to-end latency gains beyond static specialization or purely statistics-driven scheduling.

The performance breakdown reveals that the largest gains occur in regimes with high acceptance probabilities, specifically LE-LI, where D-PROMISE achieves a peak 2.45$\times$ speedup. By exploiting high target model agreement in this low-entropy setting, the engine effectively balances drafting cost against token yield to outperform all baselines. In the reasoning-intensive LE-HI regime, D-PROMISE attains 1.81$\times$ speedup, matching the robust Laptop-Only policy. This highlights a boundary case where reasoning-specialized models are near-optimal; crucially, D-PROMISE preserves these specialist benefits while retaining the flexibility to switch devices for other task types. As task entropy rises, benefits naturally diminish, though D-PROMISE maintains a lead in HE-LI (1.48$\times$). However, in the most challenging HE-HI regime, all methods converge close to the Edge-Only baseline. This indicates that when high uncertainty and reasoning demands coincide, low draft acceptance limits attainable acceleration regardless of the scheduling policy. Such convergence aligns with our latency model, confirming that verification overheads dominate the latency budget when speculative success becomes rare.

A critical insight arises from the contrast between D-PROMISE and Task-Agnostic SD. While Task-Agnostic SD relies on aggregate statistics ($\bar{\alpha}_i$), implicitly treating devices as generalists, this simplification obscures the true utility of heterogeneous devices (e.g., reasoning specialists). By explicitly modeling task alignment, D-PROMISE exploits these specialized advantages while maintaining robustness. Furthermore, the results demonstrate that speculative decoding is not



(a) Impact of Wireless Channel Degradation



(b) Impact of Device Load Saturation

Fig. 4: Robustness analysis under extreme conditions.

universally beneficial; significant acceleration is contingent on task feasibility. Consequently, granular task classification serves as a prerequisite for efficient deployment rather than a mere optimization. By avoiding indiscriminate speculation in low-agreement contexts, D-PROMISE achieves superior performance compared to the task-agnostic baseline.

C. Robustness Analysis

We stress-test D-PROMISE under two extreme sources of device heterogeneity—wireless channel degradation and device load saturation (Fig. 4). Across both scenario, D-PROMISE maintains the lowest latency per token, demonstrating that its per-request scheduling remains effective even when the underlying system state $\boldsymbol{S}(t)$ becomes highly unfavorable.

As the wireless bandwidth $B(t)$ decreases (Fig. 4a), communication overhead increasingly dominates the iteration cost, and speculative decoding is only beneficial if each transmission yields sufficient committed token (i.e., high $\alpha_{i,\tau}$). D-PROMISE mitigates this by adapting the device selection and configuration to preserve a favorable acceptance–overhead trade-off, rather than blindly offloading drafts when the channel is poor. Notably, when the network becomes prohibitive, D-PROMISE activates its fallback to edge-only execution, preventing the latency blow-up that arises from repeatedly paying high communication costs for low speculation gain.

When the device experiences heavy contention and the average device load $L(t)$ approaches saturation (Fig. 4b), static policies exhibit fragile behavior: pinning draft model to a fixed device amplifies congestion effects and leads to sharp latency growth. In contrast, D-PROMISE leverages the heterogeneous resource pool to dynamically steer requests away from overloaded nodes and toward devices with available capacity, thereby avoiding the most severe saturation regimes. This adaptive load-aware behavior and fall-back mechanism result in smoother degradation and consistently lower latency than baselines, even under near-saturation conditions. Overall, Fig. 4 confirms that D-PROMISE is robust to both communication-limited and computation-limited extremes, validating its resilience against severe runtime heterogeneity.

V. Conclusion

In this paper, we proposed D-PROMISE, a request-level framework designed to accelerate distributed speculative decoding by effectively coordinating task characteristics with heterogeneous edge resources. Crucially, our analysis reveals that speculative decoding is not universally beneficial; significant acceleration is realized only under specific task regimes, making granular task classification a prerequisite for efficient deployment. To address this, D-PROMISE introduces a novel task profiling taxonomy based on task entropy and reasoning intensity to capture the distinct preferences of draft models. Trace-driven evaluations demonstrate that D-PROMISE achieves an average speedup of 1.70 over auto-regressive inference and significantly outperforms task-agnostic speculative decoding with up to 45% performance gain. Furthermore, robustness analysis confirms that D-PROMISE maintains low latency even under extreme conditions.

References

[1] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[2] Y. Leviathan *et al.*, "Fast inference from transformers via speculative decoding," in *ICML*, 2023.

[3] C. Chen, S. Borgeaud, G. Schmitt, H. Lochner, K. Silver, L. Weidinger *et al.*, "Accelerating large language model decoding with speculative sampling," *arXiv preprint arXiv:2302.01318*, 2023.

[4] D. Ge, J. Gao, Q. Jiang, Y. Feng, and W. Ji, "Automatic task detection and heterogeneous llm speculative decoding," *arXiv preprint arXiv:2505.08600*, 2025.

[5] T. Su, M. Zhang, and G. He, "Entropy-aware speculative decoding toward improved llm reasoning," *arXiv preprint arXiv:2512.23765*, 2025.

[6] Y. Fu, R. Ge, Z. Shao, Z. Deng, and H. Zhang, "Scaling speculative decoding with lookahead reasoning," *arXiv preprint arXiv:2506.19830*, 2025.

[7] D. Xu, W. Yin, H. Zhang, X. Jin, Y. Zhang, S. Wei, M. Xu, and X. Liu, "Edgellm: Fast on-device llm inference with speculative decoding," *IEEE Transactions on Mobile Computing*, 2024.

[8] Y. Zhang *et al.*, "Clone: Customizing llms for efficient latency-aware inference at the edge," *arXiv preprint arXiv:2506.02847*, 2025.

[9] H. Xia *et al.*, "Spec-bench: A comprehensive benchmark for speculative decoding," *arXiv preprint arXiv:2401.07851*, 2024.

[10] Y. Li *et al.*, "Eagle: Speculative sampling requires rethinking feature uncertainty," in *ICML*, 2024.

[11] Z. Liu *et al.*, "Mobilellm: Optimizing sub-billion parameter language models for on-device use cases," *arXiv preprint arXiv:2402.14905*, 2024.

[12] Databricks, "Introducing llama2-70b-chat with mosaicml inference," 2023, databricks Blog.

[13] G. Developers, "Optimizing llm inference for minimal latency with vllm," 2024, google Developer Forums.

[14] D. Lo, L. B. Cheng, R. Govindaraju, P. Ranganathan, and C. Kozyrakis, "Heracles: Improving resource efficiency at scale," in *Proc. ACM/IEEE ISCA*, 2015, pp. 450–462.